PAGE NO. :
DATE : / /

'C' Programming

→ **What is Programming :-** Programming is the Process of creating instruction for a Computer to perform Specific task. it involves writting code in a Programming language to develop Software, application and System. or where User write a code in a language that Computer can understand known as Programming.

→ **Introduction about 'c' language :-** 'C' is a general purpose Programming language. It is created by 'Dennis Ritchie' in 1972 at Bell lebortries. This language known early as a 'B' language. 'c' language is developed / created for making unix OS. 'c' language fully associated with unix. It is also known as fundamental language. Mean when newly person don't to know what is Programming language then 'c' language are occur to understand How to Program is write and debug.

→ **need things to Write & Run Code :-**

1. .exe file
2. IDE

PAGE NO.:
DATE: / /

- Key features of C :- Efficiancy, Portability, Rich library, modularity, flexibility.

→ First 'C' Program

ex:- #include <stdio.h>

int main () {
printf ("Hello World : \n");
return 0;
}

→ Application of 'C' :- System Programming
Appliation development
Compiler design
Embedded System

• #include <stdio.h> :- It is header file in 'C' language. It is a preprocessor directive that tells the compiler to include the Standard input output library before compiling the program. This library typically known as the Standard I/O library and provides functionalities for input and output operation such as reading from the keyboard and writing to the console.

key functionalities provides by 'Stdio.h'

PAGE NO.:
DATE: / /

- Printf():- It is a in build function in Stdio.h library used to Print formatted output to the Console.
- Scanf():- This function is used to Read formatted text/input from the Console.
- getChar():- This function is used to read a single char from the Console.
- putchar():- It is used to Write Single char. to the Console.

→ int main():- This function in 'c' language is the entry point. When 'c' program execute, the runtime System starts by calling this function.

→ Statement in 'C':- A Computer program is a list of 'instruction' to be executed by Computer. These programming instruction are called statements.
It is important that you end the statement with a semicolon;

→ '\n':- To insert a new line, you can use the \n character.

A
13

PAGE NO.:
DATE: / /

- Comments :- Comments are used to explain our code and make it more readable. Comments are non-executable part in any language.

There are 2 type of comment.

1. Single line Comment :- it is start with two forword Slash '//'

ex:-

```
// This is Comment
```

2. multi line Comment :- it is Starts with /* and end with */

ex:-

```
/* This is
multi line
comment */
```

→ Variable :- Variable are Container for storing data Value or a named Storage location in memory that can hold a value. Variable are allow you to store, modify and retrive data during the execution of the Program.

Syntex :-

```
int age = 18;
```

type — int
name / identifier — age
value / literal — 18

PAGE NO.:
DATE: / /

→ Variable Scope :- The Scope of variable determine where it can be accessed in the code.

- local Variable :- local type variable are declared inside a function or a block. it can be used within it we can't access outside of the function or block.

- Global Variable :- Global type Variable are declare outside all function or block. we can also use this Variable inside or outside of a function or block

→ Rule for naming Variable

- Variable name can contain letter, digit and underscore
- Variable name must start with underscore and letter
- Variable name are case sensitive
- Variable name cannot contain any whitespace or a special character like :- #, @, ! etc.
- Reserved word cannot be used as Variable name.

"E" Character "C" "S" "age" 10.5f

PAGE NO.:
DATE: 9.56 7/2

10.2

- **Format Specifier** :- A formate Specifier is a placeholder used within a string to indicate where and how to display a variable's value.
Format Specifier are used in input/output function like 'printf' and 'scanf' to handle different data type.

→ **Common formate Specifier**

- %d or %i :- Used for integer
- %f :- Used for floating number
- %lf :- Used for double precisio float num.
- %c :- Used for single Charactor
- %s :- Used for string charactor
- %u :- Used for unsigned integer
- %p :- Used to print pointer address
- %% :- Used to print percentage sign

Ex.-
```
# include <Stdio.h>
int main () {
    int a = 10;
    float f = 5.0f;
    double d = 10.759;
    char c = 'b';
    char string[ ] = "HelloWorld";

    Printf ("%d \n", a);
           ("%F \n", f);
           ("%lf \n", d);
```

```
printf ("%c \n", c);
printf ("%s \n", string);
    return 0;
}
```

With Scanf function format Specifier.

ex:-

```
#include <stdio.h>
int main () {
    int i;
    float f;
    double d;
    char c;
    char string [10];

    Scanf ("%d", &i);
    Scanf ("%f", &f);
    Scanf ("%.1f", &d);
    Sanf ("%c", &c);
    Scanf ("%s", &string);
```

Syntex:-

```
int a = 5;          OR
int a=5, b=6, c=7;  OR
int x, y, z;
    x = y = z = 8;
Printf ("%d", a);
Printf ("%d", x+y+z);
```

[1, 2, 3, 4, 5] → 7

PAGE NO:
DATE: / /

→ Need to code in C :-

- MinGW (GCC Compiler
- VSCode (IDE)

Note:- 'C' program file save with '.c' extension

→ How to Run 'C' Program file

- open terminal write gcc filename.c
- after write ./a.exe in window
  ./a.out in linux or mac.

→ Data types in 'C' language

- Primary data types
  - int (4 bytes)
  - float (4 bytes)
  - double (8 bytes)
  - char (1 byte)
  - void →
- Derived data type
  - Array
  - Pointer
  - Structure
  - Union
  - function
- Enumeration data type
  - enum
- User define data type
  - typedef.

PAGE NO.:
DATE: / /

→ Type Casting / Type Conversion :- When you want to convert one data type to another data type then type casting / type conversion are occure.

There are two type of Conversion in 'C'
1. implicit Conversion (automatically)
2. Explicit Conversion (manually)

1. Implicit :- Implicit Conversion is done automatically by the compiler when you assign a value of one type to another type.

ex:-

1. float a = 9;
   Print ("%of", a);
   output = 9.0

2. int i = 9.990;
   printf("%d", i);
   output :- 9

2. Explicit Conversion :- Explicit conversion is done manually by placing the type in ( ) parenthesis in front of the value.

ex:-

1. int a = 5;
   int b = 2;
   ~~float c = a/b;~~
   ~~printf ("%of", d);~~
   float d = float (a/b);

```
1. int a = 5;
   int b = 6;
   float d = (float) a/b;
   printf ("%f", d);
```

⇒ **Constants** :- When We Want to declare a Variable Whose Value Can't be Change during execution of the program We will Use 'Const' keyWord.

A Constant is a name given to a Variable Whose Value cannot be change once it is declared.

ex:-

```
Const int roll_no = 15;
roll_no = 10;        //error
```

'Const' is a keyWord in 'C' language Which meaans once a Const Variable is declare that value Can not be change during the execution of Program.

⇒ **Operators** :- An operator are Special Symbole that perform Special operation between Variable or operands.

There are 5 types of operator

1. **Arithmatic operator** :- These type of operator are used to perform Mathmatical operation between Variable and operands.

a = 10
b = a++
a = 11

a = a+1
a+ = 10
a = 10

+, -, *, /, %, ++, --

2. Assignment operation :- These type of operator are use to assign and updates a value into Variable.

=, +=, -=, *=, /=, %=, &=, |=, ^=, >>=, <<= etc.

3. Relational/Comparision operator :- These type of operator is used to compare two Variable & operands.

==, !=, >, <, >=, <=

4. logical operator :- These type of variab operator is used give output in boolean form like : true & false

&& → And
|| → OR
! → not

PAGE NO.:
DATE: / /

→ **Boolean data type in 'c'**:- In c language, the boolean datatype is not a part of the original language Specification (c89 / c90).
However, c99 introduce a Standard way to work with boolean values by including the 'stdbool.h' header file.

stdbool.h:- It is Standard boolean type header file which is introduced in 'c99' package. It is used for find the "values of condition are true or false. Mean boolean type datatype are return values in true and false.

- once the 'stdbool.h' header file is included, you can declare variables of bool type.

ex:-

```
#include<stdio.h>
#include <stdbool.h>
    int main() {
        bool isSunny = true;
        bool isRaining = false;
        if (isSunny) {
            printf ("It's Sunny today \n");
        }
        if (isRaining) {
            printf ("It's not Raining today");
        }
        return 0;
}
```

PAGE NO.:
DATE: / /

Note:- When we trying to print boolean type values it is return output in 1 and 0 form. Whereas 1 indicate to true and 0 indicate to false.

ex:-

```
#include <stdio.h>
#include <stdbool.h>
    int main() {
        bool isSunny = true;
        bool isRaining = False;

        printf("%d", isSunny);
        printf("%d", isRainingy);
    }
```

You must use "%d" format specifier to print boolean values.

⇒ 'c' language Conditions and if else Statements:-

Conditional Statements in c are used to perform different actions based on different Conditions.

The most common conditional Statement are.

1. if
2. if - else
3. if-else if-else
4. switch

PAGE NO.:
DATE: / /

- **The 'if' Statment :-** The 'if' Statement is used to test a Condition. If the Condition is evaluates true, the block of code inside 'if' Statement is executed.

Syntax:-

```
if (condition) {
    // code to execute
}
```

Q. Write a Program to Print a no. is positive or not

ex:-

```
# include <stdio.h>
int main () {
    int number = 10;
    if (number > 0) {
        printf (" The number
                is positive");
    }
}
```

- **The 'if-else' Statement :-** The 'if-else' Statement allows you to execute one block of code if the condition is true and another block of code if the condition is false.

PAGE NO.:
DATE: / /

Syntex:-

```
if (condition) {
    // code to execute
} else {
    // code to execute
}
```

Q. Write a Program to a Person Who can drive When age is greater than 18 otherwise can't drive.

```
#include <stdio.h>
int main() {
    int age = 18;
    if (age > 18) {
        printf("He can drive");
    } else {
        printf("He can't drive");
    }
}
```

Note:- When we create a conditional statement with else part it will always execute

- The if else-if else :- The if else-if else statement allows you to check multiple conditions.

Syntex:-

if else if else

PAGE NO.:
DATE: / /

```
if (condition) {
        // code to execute
} else if (condition) {
        // code to execute
} else {
        // code to execute
}
```

Q. Write a Program to find Which type of number passed by the user.

```
#include <stdio.h>
int main() {
        int number = -10;
    if (number > 0) {
        printf ("number is positive");
    } else if (number < 0) {
        printf ("number is negative");
    } else {
        printf ("number is zero");
    }
}
```

- Conditional 'Ternary' operator:- The conditional ternary operator is a shorthand way to write 'if-else' statement.

Syntax:-

Condition ? expression_if_true : exp._if_false

PAGE NO.:
DATE: / /

Q. Write a program to find Maximum Value.

```
#include <stdio.h>
int main () {
    int a = 10;
    int b = 20;
int max = (a>b) ? a : b;
```

- 'Switch' Statements :- The switch statement is used to select one of many block of code to be executed.

Syntax.

```
switch (expression) {
    case value1 :
        // code
        break;
    Case value2 :
        // code
        break;
    default :
        // code
}
```

Q. Write a Program to print a message according to grade.

ex:-

```
#include <Stdio.h>
int main() {
    char grade = 'B';
    switch (grade) {
```

PAGE NO.:
DATE: / /

```
case 'A':
        Printf ("Excellent \n");
        break;
case 'B':
        printf ("Well done \n");
        break;
case 'c':
        printf ("You passed \n");
        break;
case 'd':
        printf ("Better try again \n");
        break;
default:
        printf ("Invalid grades \n");
    }
}
```

- The switch expression is evaluated once
- The value of expression is compare with the value of each case
- If there is match, the associated block of code is executed.
- The 'break' statement breaks out of the switch block and stop the execution
- The default statement is optional, and specifies some code to run if there is no case match. This statement work like else statment.

PAGE NO.:
DATE: / /

→ **Scanf ()** :- The 'Scanf ()' in c language are used to take input from user or Standard input like : keyboard.

Scanf() function is part of the standard input output library 'stdio.h'.

The Scanf function reads input according to the formate specifier and store them in the corresponding variable address.

1. **Reading an integer value** :-

```
Printf (" enter integer value ");
scanf ("%d", &num);
```

2. **Reading an floating value** :-

```
Printf (" enter floating value ");
Scanf ("%f", &f1);
```

3. **Reading a charactor value** :-

```
Print (" enter a character ");
scanf ("%c", &ch);
```

4. **Reading an String value**:

```
Printf (" enter a String value ");
Scanf ("%s", str);
```

DATE: / /

Q. Write a Program to take input from User and print that value on Console.

ex1-

```
#include <stdio.h>
int main() {
    int num;
    float fl;
    char ch;
    char str[100];

    printf("enter a no");
    scanf("%d", &num);
    printf("value is %d\n", num);

    printf("enter a value");
    scanf("%f", &fl);
    printf("value is %f\n", fl);

    printf("enter a character");
    Scanf("%c", &ch);
    print("entered character %c\n", ch);

    Printf("enter a String");
    Scanf("%s", str);
    printf("string is %s\n", str);

    return 0;
}
```

PAGE NO.:
DATE: / /

→ Loops :- Loops are used to execute block of code repeatedly as long as a specified condition are met. loops are handy because they save time, reduce code and they make code more readable.

In 'c' language 2 types of loop.

1. Top-tested loop :-
   (i) While loop
   (ii) for loop

2. Bottom-tested loop :-
   (i) do-while loop

While-loop :- A while loop in 'c' is a control flow statement that allow code to be executed repeatedly based on given boolean condition. The code inside the loop runs as long as the condition evaluates to true.

Syntax:-

```
i=0
while (condition) {
    // code
}
```

PAGE NO.:
DATE: / /

→ Step to Use While loop:

- Initialization:- Setup / give a value where to start a while loop.
- Condition :- The loop continues as long as this condition is true.
- Code block :- The statement to execute each iteration
- Update:- Update the value using increment/decrement operator

Q.1 Write a Program to print 1 to 10 no. using while loop.

```
#include <stdio.h>
int main() {
    int i = 0;          // Initialization
    while (i <= 10) {   // condition
        Printf("%d \n", i);  //code
        i++;            // update
    }
    return 0;
}
```

Note:- If the condition never becomes false, the loop runs forever know as infinite loop.

PAGE NO :
DATE / /

'Break Statement' :- You can use 'break' keyword to exit the loop if specified condition is met.

```
int i=0
While (i<=10){
    if (i==5){
        break;
    }
    printf("%d\n", i);
    i++;
}
}
```

⇒ do-While loop :- A do-While loop in 'c' is similar to a While loop, but with one key difference, the do-While loop guarantees that the code block inside the loop will execute at least once, even if the condition is false on the first check.

Syntax:-

```
do {
    // code
} While (condition);
```

- Step to run do-While loop.
  - Initialization
  - Code Block
  - Update

PAGE NO.:
DATE: / /

- Condition check.

Q. Write a Program to Print 1 to 10 no. using do-while loop.

```
#include <stdio.h>
int main() {
    int i = 0;          // initialization
    do {
        printf("%d \n", i);   //code
        i++;                  // update
    }
    while (i < 10);
    return 0;
}
```

⇒ For loop :- A for loop in 'c' is control flow statment that allow code to be executed repeatedly based on a condition. It is typically used when the no of iteration is known beforehand.

Syntex:

```
int i;
for (Initialization; Condition; Update) {
        // code
}
```

Q. Write a program to print 1 to 10 no. using for loop.

```
#include <stdio.h>
int main(){
    int i;
    for(i=0; i<10; i++){
        printf("%d\n", i);
    }
    return 0;
}
```

→ Nested for loop :- When one loop run using another loop known as nested for loop.

Nested for loop is used for more complex task, such as processing multi-dimensional array, iterating over collections and performing repeated calculations.

Q. Write a program to print a 5 × 5 Matrix.

```
#include <stdio.h>
int main(){
    int i, j;
    for(i=1; i<=5; i++){
        for(j=1; j<=5; j++){
            printf("%d\n", i*j);
        }
        printf("\n");
    }
```

```
return 0;
}
```

⇒ **Continue Statment** :- The Continue Statement break one iteration in the loop, if a Specified Condition is met and Continue with the next iteration in the loop.

ex:-

```
int i;
for(i=1; i<=10; i++){
    if (i=7){
        continue;
    }
    printf("%d\n", i);
}
return 0;
}
```

output:

```
1
2
3
4
5
6
8
9
10
```

```
1                       1
1 2                   2 1
1 2 3               3 2 1
1 2 3 4           4 3 2 1
1 2 3 4 5       4 3 2 1
```

PAGE NO.:
DATE: / /

Arrays:- An Arrays is a Collection datatype it is used to store multiple same type of elements into a single variable in contiguous memory location.

- Arrays Vaules Can be access by its index no. the array index no starts with 0 and so on.
- Arrays are fixed in size (The size of an array are defined at time of declaration. once an array is declare with its size we can not change its size dynamically)

→ **Declaration and Initialization :-**

```
int number [10];                    // Uninitialized
int number [5] = {1, 2, 3, 4, 5};   //fully initia
int number [5] = {1, 2}             // partially Initialize
int numbe [] = {1, 2, 3, 4, 5, 6}
```

ex:-

```
# include <stdio.h>
int main(){
    int number [5] = {1, 2, 3, 4, 5};
    // Accessing Array element
    Printf(" first element : %d \n",
                        number [0]);
    printf(" second element : %d \n",
                        number [1]);
```

PAGE NO.:
DATE: / /

```
//Modifying Array element
number[2] = 10;
printf("Array after modified: %d\n"
                                number);
return 0;
}
```

→ Array with loop:-

```
#include <stdio.h>
int main() {
    int number[] = {10, 20, 30, 40, 50};

    for (int i=0; i<5; i++) {
        printf("Element at index %d:
                %d\n", i, number[i]);
    }
    return 0;
}
```

Ex:

```
#include <stdio.h>
int main() {
    int arr[5];
    arr[0] = 5;
    arr[1] = 9;
    arr[2] = 7;
    arr[3] = 19;
    arr[4] = 20;
for (int i=0; i<5; i++) {
    printf("%d\n", arr[i]);
}
return 0;
}
```

PAGE NO.:
DATE: / /

→ Get Array Size or length:

To get the size of Array, We can use Sizeof operator.

Sizeof operator:- The Sizeof operator is used to determine the size of Variabl or datatype.

A Sizeof operator return a size of array into bytes.

Ex:-
```
# include <stdio.h>
int main(){
    int arr[5] = {1, 2, 3, 4, 5};

    printf("%d\n Size of array ", sizeof(arr));
    printf("%d\n Size of one array element ", sizeof(arr[
    return 0;
}
```

output 20
4

→ To find the exact length of the arry to apply this formula.

Ex:- int length = Sizeof(arr) / sizeof(arr[

```
#include <stdio.h>
int main(){
    int arr[5] = {1, 2, 3, 4, 5};
    int length = sizeof(arr) / sizeof(arr[0]);
    printf("Size of array is %d \n", length);
    return 0;
}
```

output : 5.

Q.1. Write a Program to Calculate Avg. of array elements.

ex:
```
#include <stdio.h>
int main(){
    int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int length = sizeof(arr) / sizeof(arr[0]);
    int sum = 0, avg;
    for(int i=0; i<length; i++){
        sum += arr[i];
    }
    avg = sum / length;

    printf("The avg. of array element is %d
                \n", avg);
    return 0;
}
```

PAGE NO.:
DATE: / /

Practice Question

Q1 Write a Program to ~~find~~ find lowest no. in the array.

Q.2 Write a program to Search an element are available or not in Array.

→ **Multi-Dimensional Arrays:-** A multi-Dimensinal Array is an array of arrays. It allow you to Store data in more than one dimension, typically organize in a grid-like structure. This Concept extends the idea of a one-dimensiond array to two o more dimensions.

- **2D / Two-Dimensional Array :-** A 2D Array Can be designed as a table with row and columns.
- 2D Array elements are accessed using two indices. one for the row and one for the col.
- Creating 2D Array

```
• int arr [3] [3] = {{1,2,3}, {4,5,6}, {7,8,9}}
• int arr [3] [3];
    arr [0] [0] = 1.
    arr [0] [1] = 2.
    arr [0] [2] = 3
    arr [1] [0] = 4
    arr [1] [1] = 5
    arr [1] [2] = 6
    arr [2] [0] = 7
    arr [2] [1] = 8
    arr [2] [2] = 9
```

Ex: #include <stdio.h>
int main(){
int arr[3][4] = {
{1, 2, 3, 4},
{5, 6, 7, 8},
{9, 10, 11, 12}
}

→ Access 2D Array elements

printf("Elements at matrix[0][0]: %d\n", arr[0][0]);
printf("Element at matrix[1][2]: %d\n", arr[1][2]);

→ print 2D Array

```
for(int i=0; i<3; i++){
    for(int j=0; j<4; j++){
        printf("%d ", arr[i][j]);
    }
    printf("\n");
}
return 0;
}
```

→ Change elements in 2D Array :-

```
int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
matrix[0][1] = 9;
printf("%d", matrix[0][1]);
```

PAGE NO.:
DATE: / /

3D Array:- A 3D array in c is essentially an array of arrays. It can be thought of as a cube of data where each element is accessed using three indices: one for the first dimension, one for the second, and one for the third.

→ **Declaring 3D Array**

```
int arr[2][3][4];
```

[2] → no. of array
[3] → Each array contain 3 rows
[4] → Each row contain 4 column

→ **initializing 3D Array:-**

ex:-
```
#include <stdio.h>
int main(){
    int arr[2][3][4]={ {
                        {1, 2, 3, 4},
                        {5, 6, 7, 8},
                        {9, 10, 11, 12}
                      },
                      {
                        {13, 14, 15, 16},
                        {17, 18, 19, 20},
                        {21, 22, 23, 24}
                      }
                    };
```

PAGE NO.:
DATE: / /

// Access & print the 3D Array element

```
for (int i=0; i<2; i++){
    for (int j=0; j<3; j++) {
        for (int k=0; k<4; k++) {
            printf(" array [%d] [%d] [%d]
                = %d \n", i, j, k,
                arr [i] [j] [k] );
        }
    }
}
return 0;
}
```

→ Change/modify 3D Array element

```
int newvalue = 42;
arr [1] [2] [3] = newvalue;

    printf("\n modified array \n");
for(int i=0; i<2; i++){
    for (int j=0; j<3; j++){
        for (int k=0; k<4; k++) {
            printf (" array [%d] [%d] [%d] =
                %d \n", i, j, k, arr
                [i] [j] [k]);
        }
    }
}
return 0;
}
```

PAGE NO.:
DATE: / /

String :- A String is a Sequence of character Stored in memory. Which is used to represent text.

'C' Programming doesn't have any String data type so here We declare string type value using Character array.

String are implemented as array of character With a Special null terminator to mark the end of the ~~Set~~ String.

→ Key Characteristics of String :-

- Character Array :- A String is a ~~for~~ represented as a Contiguous block of memory, Which is Used to ~~repres~~ represent Sequence of character, and this block is typically defined using a character array. The last character of the array must be the null character '\0' to Signify the end of the String.

- Null-Terminator/character :- String in c are terminated by a Special character called the null terminator ('\0'). This null character indicates the end of the String and is used by various functions to determine where the string ends.

PAGE NO :
DATE : / /

- no-length-information :- Unlike some other programming language, C string do not store their length explicitly. The length of the string is determine by finding the null terminator.

* String Declaration & initialization :-

1. Using character Array :-

ex!.

```
char str1[] = "Hello World";
```

2. Using character Array with Explicit initialization

ex:-

```
Char str2[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

3. Using String literals with pointer

ex!.

```
Const char *str3 = "Hello, World";
```

Note :- A Space (" ") is also a character in String. String Chacter always denote in (" ") double quote.

ex:-

```
# include <stdio.h>
int main () {
    Char str1[] = "Hello World";
    Char str2[] = {'H', 'e', 'l', 'l', o', '\0'};
Const Char *str3 = "Hello Jhon";
```

```
printf (" String 1 is : %s \n", str1 );
Printf (" String 2 is : %s \n ", str2 );
printf (" String 3 is : %s \n", str3 );

    return 0;
}
```

→ 'C' Programming provides a ~~String~~ Standard library <String.H> which is used to made some operation on String type value <String.h> library include. Some function to manipulate or update String.

String function

- Strlen() : it is used to Compute / count length of the String.
- Strcpy () : it is used to return a copy of previous string.
- Strcat () : it is used to Concateness (append) one String to another String.
- Strcmp () : it is used to Compare two String Values

Note:- When we declare String and not initialize its value then it is mandatory to give String value character size:

PAGE NO.:
DATE: / /

ex:- String with initialization
Char Str1 [ ] = "Hello World"

String without initialization
Char str2 [50];

→ A Program with String function.

```
#include <stdio.h>
#include <string.h>
    int main() {

// for ~~cha~~ Strlen & strcpy function.
    char str1 [ ] = "Hello World";
    char str2 [50];

int size_t length = Strlen(str1);
    printf("length of str1 is : %zu\n", length);

    Strcpy(str2, str1);
    printf("str2 value is : %s\n", str2);

// For strcat function
    char str3 [ ] = "Wide Web";

    Strcat(str1, str3);
    printf("Concatenated String is : %s\n",
                str1);
```

PAGE NO. :
DATE : / /

// For String Comparision

```
Char Str4 [] = "apple";
Char Str5 [] = "Banana";

int result = Strcmp (Str4, Str5);
if (result < 0) {
  printf ("'%s' is less than '%s'\n",
                    Str4, Str5);
} else if (result > 0) {
  printf ("'%s' is greater than '%s'\n",
                    Str4, Str5);
} else {
  printf ("'%s' is equal to '%s'\n",
                    Str4, Str5);
}
return 0;
}
```

& ptr = &

PAGE NO.:
DATE: / /

* Pointer :- A pointer is a variable that stores memory address of another variable as its value. A pointer variable points to a datatype of the same type and it is created with '*' operator. A pointer holds the location where the data is stored in memory. This allows you to indirectly access and manipulate the data.

-> Declaration of pointer :- Here, we declare a pointer in 2 ways.

- int* no;
- int *no;

-> initialization :- A pointer must be initialized to point to a valid memory address, typically the address of another variable. This is done using the address-of operator '&'.

ex:- int age = 20;
int *ptr = &age;

=> Dereference :- Dereferencing a pointer means accessing the value at the address the pointer is pointing to, using the '*' operator.

ex:
int result = *ptr;

PAGE NO.:
DATE: / /

-> Pointer arithmatic :- You can perform arithmatic operation on pointers, such as incrementing and decrementing them. This is useful for iterating through arrays.

ex:- ptr ++;

ex:-

```
#include <stdio.h>
int main (){
    int no = 42;
    int *ptr = &no;

// Print the address stored in pointer
    printf("address of ~~point~~ no: %p \n", ptr);

// Print the value of no. using Pointer
    printf(" value of no is: %d \n", *ptr); \n

// update the value of no. using Pointer
    *ptr = 55;
    printf(" updated value of no. is: %d\n", no);

// With array

    int arr[] = {10, 20, 30, 40};
    int *arr_ptr = arr;

    // print array element using pointer
```

```
for(int i=0; i<4; i++){
    printf("arr[%d] = %d\n", i, *(arr_ptr
                                  +i));
    }
    return 0;
}
```

→ Why We Use Pointer :-

- Dynamic memory management :- pointer are essential for dynamically allocating and freeing memory using function like "malloc", "calloc", "realloc", and "free".

- Array & String manipulation :- pointer are used to efficiently work with array & string allowing for fast & flexible access

- Function Arguments :- pointers allows functions to modify the values of arguments or pass large data structure without copying them.

- Efficient data Structure :- They are used to create complex data structure such as linked list, trees and graphs.

PAGE NO. :
DATE : / /

→ Dereferencing :- Dereferencing a pointer means acces
The values stored at the memory
address that the pointer holds. I
'c' language We use '*' operator to dereference
a pointer.

ex:- int value = *ptr;

Here, *ptr gives you the value of x because 'ptr'
points to 'x'.

ex- #include <stdio.h>
int main () {
int x = 10;
int *ptr = &x;
// Dereferencing
printf ("value of x is: %d\n", *ptr);

// modify using pointer
*ptr = 20;
printf ("updated value of x is: %d\n" *p

return 0;
}

→ Pointer & Array :-

- Array name as a pointer :- The name of an
array in c represent
a pointer to its

first element. When you use the array name in an expression, it is treated as a pointer to the first element of the array.

ex:

```
int myNumber[4] = {1, 2, 3, 4};

//get the memory address of the array
printf("%p\n", myNumber);

//get the memory address of first array element
printf("%p\n", &myNumber[0]);
```

output: Same address.

The memory address of the first element is the same as the name of the array.

```
//get the value of first element in array
//using pointer
printf("%d\n", *myNumber);
printf("%d\n", *(myNumber+1)); for 2nd
printf("%d\n", *(myNumber+2)); for 3rd
```

→ Array element with loop using pointer

ex:

```
#include <stdio.h>
int main(){
    int arr[4] = {5, 9, 10, 11};
    int *ptr = arr;

    for(int i=0; i<4; i++){
        printf("%d\n", *(ptr+i));
    }
    return 0;
}
```

PAGE NO.:
DATE: / /

→ Change array element using pointer :-

```
*arr = 13;      // 0th index
*(arr+1) = 15;  // 1th index
```

⟹ Memory Management function in Pointer :-

Memory management function in c are used to handle dynamic memory allocation and deallocati These function are essential when working with pointers and managing memory during program execution They allow you to allocate, resize and free memory dynamically, as opposite to using static memory allocation, which is done at compile time.

There are 4 type of Memory management function

1. malloc
2. Calloc
3. realloc
4. Free.

1. malloc() :- The malloc() function is a standard library function and it is used for dynamic memory allocation. It is stand for memory allocation. and it is part of the <stdlib.h> library.

- malloc function allocate a specified number of bytes memory during runtime. This is useful when the total amount of memory needed cannot be determind at compile time.

PAGE NO.:
DATE: / /

- It return a pointer to the beginning of the allocated memory block. The memory block is not initialized, meaning it contains "garbage" values.
- If memory allocation fails (eg. due to insufficient memory), the malloc function return 'NULL' values.

Syntax:-

```
void* malloc(size_t size);
```

- Size:- The no. of memory size allocation in bytes.
- size_t :- It is a unsigned integer datatype which is defined in <stddef.h> library.

→ Steps to use malloc() function

- include <stdlib.h> header file
- Allocate memory: call (malloc) with no. of bytes memory you need
- Check for success: always check if the return pointer is 'null' to handle allocation fails.
- use the memory: declare the 'void*' pointer to the desired type and use it.
- free the memory:- when you are done with allocation of memory, use 'free' to de-allocate it. and prevent memory leaks.

PAGE NO.:
DATE: / /

Q:

```
#include <stdio.h>
#include <stdlib.h>
int main (){
    int *arr;
    int n = 5;

// Allocate memory for array of 5 integer
    arr = (int*) malloc (n* sizeof(int));

// check if memory allocation Dos Successful
    if (arr == NULL){
        fprintf (stderr, " memory allocation failed \n
        return 1;
    }

// initialize the allocated memory
    for (int i=0; i<n; i++){
        arr[i] = i * 10;
    }

    Printf (" Array elements: \n" );
    for (int i=0; i<n; i++){
        printf ("%d ", arr[i]);
    }
    printf ("\n");

// free the allocated memory
    free (arr);

    return 0;
}
```

PAGE NO.:
DATE: / /

2. Calloc():- The calloc() function is used for dynamic memory allocation, similar to 'malloc', but with an additional feature to initialize the allocated memory to zero. This is particularly useful when working with array & structure where initialization is important. It return a pointer to the beginning of the allocated memory block if the allocation is failed, it return NULL.

Syntax:

```
void* calloc(size_t num, size_t size);
```

- num:- number of element to allocate.
- size:- The size of each elements in bytes.

-> Steps to use calloc() function

- include header file
- Allocate memory :- With the no. of element & the size of each element.
- Check for success.
- Use the memory
- free the memory

PAGE NO.:
DATE: / /

ex:-
```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int *arr;
    int n=5;

//allocate memory for an array of 5 int & initialize to 0
    arr = (int*) calloc (n, sizeof(int));

//check if memory allocation was successful
    if (arr == NULL){
        fprintf (stderr, "memory allocation failed\n");
        return 1;
    }

//print array element to show they initialized to zero
    for (int i=0; i<5; i++){
        printf ("%d ", arr[i]);
    }
    Printf ("\n");

    free (arr);

    return 0;
}
```

PAGE NO.:
DATE: / /

3. realloc() :- The realloc() function in 'C' is used to resize an already allocated memory block. It is a crucial tool in dynamic memory management. It is allowing us to adjest the size of a memory block while preserving its existing content.

Syntax:- void* realloc(void* ptr, size_t size);

- ptr :- It is a pointer to the memory block previously allocated by 'malloc()', 'calloc()', or 'realloc()'. If ptr is null then 'realloc()' function behaves like 'malloc()' function and allocates a new block of memory

- Size:- The new size in bytes for the ~~new~~ memory block.

ex:-
```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *ptr;
    int original_size = 5;
    int new_size = 10;

    //Allocate memory for 5 integer
    ptr = (int*) malloc(orsinal_size * sizeof(int));
    if (ptr == NULL) {
        printf("memory allocated Failed\n");
        return 1;
    }
```

```
//initialize memory block
for(int i=0; i<orisinal_size; i++){
        ptr[i] = i+1;
    }

//Resize the memory block to hold 10 integer
    ptr = (int*) realloc(ptr, new_size * sizeof(in
        if (ptr == NULL){
        printf("memory re allocation is failed");
        return 1;
    }

//initialize the newly allocated memory
for(int i=0; i<new_size; i++){
        ptr[i] = i+1;
    }

// print the entire array
for(int i=0; i<new_size; i++){
        printf("%d ", ptr[i]);
    }

// free the allocated memory
        free(ptr);
    return 0;

}
```

output:

1 2 3 4 5 6 7 8 9 10

PAGE NO :
DATE : / /

- **Free()** :- The free() function is a fundamental part of memory management in C. It is used to deallocates memory that was previously allocated by 'malloc()', 'calloc()', or 'realloc()'. A free() function basically used for avoiding memory leaks and ensure efficent use of memory.

Syntex:- `Void free(ptr);`

⟹ **Function** :- A function in 'C' is a block of code that perform specific task. Using function we can break down a program into smaller parts, modular pieces that are easier to manage, understand and reuse. A function typically takes some input (called parameter/ arguments) process on it and return a result on the console.

- **function Declaration** :-

```
return_type function_name (parameter) {
        //function body
        return value;
}
```

returntype → name of the function

```
int addition(int a, int b) {
    int Sum;
    Sum = a+b;
    return Sum;
}
```

keyword (return value)

-> Function Calling :- We can call a function using name of function followed by the parenthesis with Semicolon.

ex:-

```
result= addition(25, 60);
```

- Parameter :- A parameter is variable name which is write when function is created.

ex:-

```
int addition(int a, int b) {
```

Here, int a & int b is a parameter.

- Arguments :- An arguments is a value it is pass when the function called by the user.

ex:- result = add (25, 60);

Here, 25 & 60 is argument in function.

Note:- A function is block of code which only runs when it is called.

ex. a function to print a name

```
# include <stdio.h>
  void printName (char name[]){
        printf (" Hello %s\n", name);
    }
  int main (){
        printName ("Ram");
        printName ("Shyam");
        return 0;
    }
```

output:- Hello Ram
Hello Shyam

→ a function with multiple parameter

```
# include <stdio.h>
  void myfunction (char name[], int age){
        printf (" Hello %s. You are %d
                Years old.\n", name, age);
    }
  int main (){
        myfunction ("Ram", 22);
        myfunction ("Shyam", 30);
```

output:-

Hello Ram You are 22 Year old.

→ Arrays in function

```
# include <stdio.h>
  void printArray (int arr [5]){
        for (int i=0; i<5; i++){
            printf ("%d\n", arr[i]);
        }
    }
```

```
int main (){
    int arr[5] = {1, 2, 3, 4, 5};
        printArray(arr);
        return 0;
    }
```

Note:- When you are working with multiple parameter the function call must have same no. of argument as the no. of parameter. and the argument must be passed in the same order.

→ a function with return values.

```
#include <stdio.h>
int newfunction (int x){
        return 5 + x;
    }
int main (){
    printf("result is %d \n", newfunction(15));
        return 0;
    }
```

output = 20

or

```
int addition (int a, int b){
        return a * b;
    }
int main {
int result = addition (15, 20);
    printf ("result is %d \n", result);
        return 0;
    }
```

PAGE NO.:
DATE: / /

→ Types of function :- In c language there are 2 types of function

1. Built-in function / Predefine
2. User define function.

1. Builtin function :- These type of function are already define in 'c' language library like- printf(), scanf(), sizeof() etc.

2. User-define function :- These type of function created by the user according to ~~their~~ their needs.

Q.1 Write a Program to check whether a no is prime using function.

```
#include <stdio.h>
int isPrime(int n){
    if (n<=1){
        return 0;
    }
    for(int i=2; i<=n/2; i++){
        if (n % i == 0){
            return 0;
        }
    }
    return 1;
}
```

PAGE NO :
DATE : / /

```
int main () {
    int num;
    printf ("Enter a no.");
    scanf ("%d", &num);

    if (isPrime (num)) {
        printf ("%d is a prime no.\n", num);
    } else
        printf("%d is not a prime no.\n", num);

    return 0;
}
```

Q. Write a Program to Swap Two no using a function

```
#include <stdio.h>
void swap (int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main () {
    int x, y;
    printf("Enter two no");
    scanf ("%d %d", &x, &y);

    printf("before swapping : X = %d, Y = %d\n"
                                    x, y);
    swap (&x, &y);
    printf (" after swapping : X = %d, Y = %d \n"
                                    x, y);
```

PAGE NO :
DATE : / /

Q. Program to find the greatest Common Division (GCD) using function.

```
#include <stdio.h>
int gcd(int a, int b) {
        if (b==0)
            return a;
        else
            return gcd(b, a%b);
    }
int main() {
        int num1, num2;
        printf("Enter two no");
        Scanf("%d %d", &num1, &num2);

        int result= gcd(num2, num2);
        printf("GCD of %d and %d is %d\n",
                num1, num2, result);

        return 0;
    }
```

→ Practice Question

1. WAP to find the Sum of Elements in an Array using a function
2. WAP to find the factorial of a no. using function.

PAGE NO.:
DATE: / /

C math functions:- In C programming has a list of mathematical function that allow you to perform mathematical tasks on no. To use them, you must include math.h library in you program.

math functions

1. pow() : Return the value of raised to the power
2. Sqrt() :- Square root
3. fabs() :- Return absolute value of a floating point no.
4. Ceil() :- Round up to the nearest integer
5. floor() :- Round down to the nearest integer.
6. round() :- Round to the nearest integer

Q.11 8

PAGE NO.
DATE / /

→ Structure :- A structure in C is a user-defined data type that allows you to group different types of variable into a single name. Unlike arrays, structure can store variable of different data types (integer, float, character) in a single unit.
The memory allocated to a structure is the sum of the memory required by each member of the structure. Each member has it's own memory location.

Syntax:- Struct structure-name { name of Struct

keyword

datatype no1
datatype no2 ——

};

ex:-
```
#include <stdio.h>

//Define structure
struct person {
        char name[50];
        int age;
        float height;
};
int main (){
        // Delare a variable of type struct
        struct person person1;

        //Assign value to the struct member
        person1.name = "Radha";
```

```
person1.age = 30;
person2.height = 5.0;

// Access and print the values
printf("name: %s\n", person1.name);
printf("age: %d\n", person1.age);
print("height: %f\n", person2.height);
```

→ Why use Structure:-

1. model Real-world entities
2. organize related data
3. pass Data to function
4. memory management

→ Nested Structure:- Structure in C can also be nested meaning one Structure can contain another structure as a member. This process allow even more ~~me~~ Complex data modelli

ex:
```
#include <stdio.h>

struct Address {
        char city[50];
        char State[50];
};

struct Person {
        char name[50];
        int age;
        struct Address address;
};
```

```
int main () {
        Struct person per;

            per.age = 25;
Snprintf (per.name, sizeof (per.name), "Govind");
Snprintf (per.address.city, sizeof (per.address.city),
                                        "Jaipur");
Snprintf (per.address.state, sizeof (per.address.state),
                                        "Rajasthan");

printf ("name : %s \n", per.name);
printf ("age : %d \n", per.age);
printf ("city : %s \n", per.city);
printf ("State : %s \n", per.state);

return 0;
}
```

PAGE NO.
DATE: / /

Union :- A union in c is a user-define data ty like a structure, It allows you to store different data type in the same memory location. However unlike structure where each member has its own memory space, a union allocates a single shared memory space that is used by all its member. This mean that only one member of the union can hold a value at any given time.

- All member of union share same memory location. The size of the union is determine by the size of its largest member.

Syntax:- union union_name {
    datatype d1;
    datatype d2;
};

ex. #include <stdio.h>
union Data {
    int i;
    float f;
    char str[20];
};
int main() {
    union Data data;

    data.i = 10;
    printf("data.i : %d\n", data.i);

o/p :- ~~using~~ use for find/count size of ~~data~~ & structure

PAGE NO.:
DATE:

```
data.f = 220.5;
printf("data.f : %f\n", data.f);

Snprintf(data.str, sizeof(data.str), "Hello");
printf("data.str : %s\n", data.str);

return 0;
}
```

→ Why we use Union

- Memory Efficiency
- Variant Data Structure
- interfacing with ~~Data~~ hardware.

⇒ Some example of Structure & Union

1. Array of Structure

```
#include <stdio.h>
Struct Employee {
        char name[50];
        int id;
        float salary;
};
int main(){
    Struct Employee emp[3];

    for(int i=0; i<3; i++){
        printf("Enter details of employee %d\n",
                i+1);
```

```
printf("Name : ");
Scanf("%s", emp[i].name);
printf("ID : ");
Scanf("%d", emp[i].id);
print("Salary :");
Scanf("%f", emp[i].salary);
printf("\n");
}
// Display details
printf("\n Employee %d\n");
    for(int i=0; i<3; i++){
        printf("Employee %d\n", i+1);
        printf("Employee-Name : %s\n", emp[i].nam
        printf("Employee-ID : %d\n", emp[i].id);
        printf("Employee-salary : %f\n", emp[i].salary
            printf("\n");
    }

    return 0;
}
```

→ Structure with pointer

```
# Include <Stdio.h>
int Struct Car {
        char model[50];
            int Year;
            float price;
        };
    int main() {
        Struct Car car1 = {"Toyota corolla",
                                2018, 20000.0};
```

```
struct Car * c_ptr = &car1;

// Access structure member using pointer
printf("Car Model: %s\n", c_ptr -> model);
printf("Manufacturing year: %d\n", c_ptr -> year);
printf("Price: %.2f\n", c_ptr -> Price);
return 0;
}
```

→ Some example of Union :-

- Memory Sharing in Union

```
#include <stdio.h>
union Data {
    int i;
    float f;
    char str[20];
};
int main() {
    union Data d;

    d.i = 100;
    d.f = 100.00f;
strcpy(d.str, "Hello");

    printf("Integer value: %d\n", d.i);
    printf("Float value: %f\n", d.f);
    printf("String value = %s\n", d.str);
    printf("Size of union: %lu bytes\n", sizeof(d));
return 0;
}
```

PAGE NO.:
DATE: / /

-) Union With Structure

```
#include <stdio.h>
Struct Employee{
    int empID;
    char name[50];

    Union {
        int hourlyWage;
        float monthlySalary;
    } payment;
};
int main(){
    Struct Employee emp1, emp2;

    emp1.empID = 1;
    Strcpy(emp1.name, "Shayam");
    emp2.payment.hourlyWage = 20;

    emp2.empID = 2;
    Strcpy(emp2.name, "Ram");
    emp2.payment.monthlySalary = 5000.0;

printf("Employee1: %s has hourly wages of $%d/n",
        emp1.name, emp1.empID, emp1.payment.
                                hourlyWage);

    return 0;
}
```

PAGE NO :
DATE : / /

Typedef :- The typedef keyword in C is used to create an alias or new name for an existing data-types. This can make the code easier to read and man Manage, especially when dealing with complex data-type like structure, pointer, or arrays.

Why we use typedef :-

- Simplify Complex Data-types.
- Code Readability
- portability
- Ease of use.

Syntax:-

```
typedef existing type new-type name;
```

Ex-
```
#include <stdio.h>
typedef int* Intpointer;

int main(){
    Intpointer p, q;
    int a = 10;
    p = &a;
    q = &a;
    printf("value of *p :%d\n", *p);
    printf("value of *q %d\n", *q);

    return 0;
}
```

PAGE NO.:
DATE: / /

- typedef with structure

```
#include<stdio.h>
typedef struct {
        char name[30];
        int age;
        float height;
} person;

int main(){
    person person1;
    person1.age = 20;
    person1.height = 5.9;
    Snprintf(person1.name, sizeof(person1.name),
                "Radha");
    printf("name : %s\n", person1.name);
    printf("Age : %d\n", person1.age);
    printf("height : %f\n", person1.height);
    return 0;
}
```

⇒ File Handling :- File handling in C is a way to store a data in a file and retrive a data from file when needed. This file handling provides some features like read a file, write to a file, open a file & close a file.

C provides a set of function for file handling in the <Stdio.h> library.

1. fopen() : Used to open a file
2. fclose() : Used to close an open file.
3. fprintf() : Write formatted output of a file.
4. fScanf() : Read formatted input from a file
5. fgets() : Read a string from a file.
6. fputs() : Write a string from a file.
7. Fread and fwrite() :- Read & write binary data.
8. fseek() : move the file pointer to a specific location
9. ftell() : Return the current position of the file pointer.

File Modes :- When opening a file, you specify the mode.

- "r" : open a file for reading.
- "w" : open a file for writting. if the file exist, it is truncated to zero length. if doesn't exist, a new file is created.
- "a" : open a file for appending at the end.
- "r+" : open a file for reading or writting.
- "w+" : open a file for reading or writting if the file exists
- "a+" : open a file for both reading or opending.

Ex1. Writting to file

```
#include <Stdio.h>
int main(){

    File *fpointer;
```

```
// open a file in write mode
fpointe = fopen("example.txt", "w");

    //check if file open successfully
      if (fpointer == NULL) {
          printf("error opening file!\n");
          return 1;
      }
    //write data into a file
    fprintf(fpointer, "Hello world\n");
    Fprintf(fpointer, "File content written done\n");

    //close the file)
     Fclose(fpointer);

    Printf("Data written successfully.\n");

    return 0;
}
```

CY:-2 Reading from a file

```
#include <stdio.h>
int main() {
      File *fileptr;
      char buffer[100];

      //open a file ~~for~~ in read mode
        fileptr = fopen("example.txt", "r");

      //check if file exists and open
```

```
if (fileptr == NULL) {
    printf(" file not found \n");
    return 1;
}
// Read a line from the file using fgets and print it
while (fgets(buffer, 100, fileptr)) {
    printf("%s", buffer);
}
// close the file
fclose(Fileptr);
return 0;
}
```

Ex3 Appending to a file

```
#include <stdio.h>
int main() {
    File *fileptr;

    //open file in append mode
    fileptr = ~~FILE~~ open(" filename", "a");
    if (fileptr == NULL) {
        printf(" error opening file \n");
        return 1;
    }
    //Append text to the file
    fprintf(fileptr, " Aprending new line in file \n");

    // close the file
    fclose(fileptr);
    return 0;
}
```

PAGE NO. :
DATE : / /

Ex4 ftell() & fseek()

```
#include <stdio.h>
int main(){
    File *fileptr;
    char buffer[100];

    fileptr = fopen("Filename", "r");
        if (fileptr == NULL){
            printf(" error opening file \n");
            return 1;
        }

//move the file pointer to the end of file
    fseek(fileptr, 0, SEEK_END);

//Get the Current position (size of file in bytes)
    long filesize = ftell(fileptr);
    printf(" size of file : %ld bytes \n", filesize);

// move the file pointer back to the beginning
    fseek(fileptr, 0, SEEK_SET);

//Read and print the first line
    fgets(buffer, 100, fileptr);
    printf("First line : %s", buffer);

    fclose(fileptr);
return 0;
}
```

PAGE NO.:
DATE: / /

SEEK_SET :- Start of the file
SEEK_CUR :- current position
SEEK_END :- End of the file

→ fprintf() :- This function is used to write formatted output to a file or a specified output stream. It is same as printf() function, but it allow you to specify the file or stream where the output will be written.

Syntax: int fprintf(File *stream, const char *format, ...);

→ Stderr Stream :- Stderr is the standard error output stream in C. it is used for outputing error message or diagnostic, and it is unbuffered, mean it displays output immidiatly. By default, Stderr outputs to the terminal.

Difference between Stdout or Stderr

- Stdout :- Standard output stream used by printf() function to print normal program output. It can be redirect to a file.
- Stderr :- Standard error stream used to print error message. It is typically not redirected and immidiately shown in the console.

PAGE NO.:
DATE: / /

Stream:- A stream is a sequence of character or bytes flowing from or to a device like (file, keyboard, or terminal).

Stream can be categorized into input or output stream.

- input Stream:- It Reads the data from keyboard or file.
  ex: Stdin is the standard input stream.

- output Stream:- It writes the data to a destination (screen or file). Stdout is the standard output stream.

Stream functions

1. **input function**:
   - fscanf():- Read formatted input from stream
   - fgets():- Read a string from a stream.
   - fgetc():- Read a single character from str

2. **output function**:-
   - fprintf():- Write formatted output to a stre
   - fputs():- Write a string to a stream.
   - fputc():- Write a single character to a stream.